

Cómo pensar en Lógico

El método en 5 pasos, los errores que se repiten, y los reflejos que hay que automatizar antes del parcial.

- 1 · El método: 5 pasos antes de escribir código
- 2 · Cabeza vs. cuerpo: la confusión #1
- 3 · Variables: ligadas, libres y puentes
- 4 · Generador primero, restricción después
- 5 · "Todos": forall y la doble negación
- 6 · El test de 5 segundos
- 7 · Diccionario castellano → Prolog
- 8 · Los 10 errores que se repiten
- 9 · Checklist de parcial

1 · El método

No arranques tipeando. Estos cinco pasos toman treinta segundos y te ahorran veinte minutos de dar vueltas.

1 Escribí la frase en castellano, con el "si" en el medio.

Literalmente escribila al costado de la hoja:

"X es tío de Y **si** X es hermano del padre de Y"

Lo que está **antes** del "si" es la cabeza. Lo que está **después** es el cuerpo. Cada sustantivo o verbo del lado derecho se va a convertir en una consulta.

Si la frase no te sale en castellano, el problema no es Prolog: todavía no entendiste el requerimiento.

2 Preguntate: ¿esto me lo dan o lo tengo que deducir?

Lo que te dan son **hechos** (la base de conocimientos). Lo que te piden es una **regla**. Nunca reescribas ni pises un hecho que ya existe.

ALARMA Si estás cargando hechos a mano en un ejercicio que pide "desarrollá el predicado tal", algo está mal.

3 Listá las variables y clasificalas.

Tres tipos: las que vienen de la cabeza, las que nacen adentro del cuerpo (los *puentes*), y las que no te importan (`_`).

Una variable que aparece **una sola vez** en toda la regla debería ser `_` . Una que aparece dos veces está uniendo dos consultas — verificá que sea a propósito.

4 Ordená: generador primero, restricción después.

Mirá qué consultas pueden *producir* valores (los hechos de la base) y cuáles solo pueden *verificar* (`not` , `forall` , `is` , `>` , `<` , `\=`). Las primeras van adelante.

5 Delegá antes de dar por terminado.

¿Hay un concepto del dominio con nombre propio adentro de tu regla? (*tener experiencia*, *ser abuelo*, *ganar bien*). Sacalo a un predicado aparte. La regla principal tiene que leerse casi igual que el enunciado.

2 · Cabeza vs. cuerpo

Este es el error más común y el más fácil de detectar. La **cabeza** es lo que estás definiendo; el **cuerpo** es la evidencia de la que lo deducís.

MAL

```
empleado(Sector, Nombre, Puesto) :-  
    trabajaEn(Sector, Nombre).
```

BIEN

```
trabajaEn(Sector, Nombre) :-  
    empleado(Sector, Nombre, _).
```

"Es empleado si trabaja en" — pero `empleado/3` son los hechos que vos cargaste. No podés definir con una regla algo que ya tenés como hecho.

"Trabaja en si es empleado" — `trabajaEn` es lo que las consultas preguntan, o sea lo que querés *concluir*.

El test: leé tu regla en voz alta con el "si" en el medio. Si suena al revés, está al revés.

"*Empleado si trabaja en*" no tiene sentido; "*trabaja en si es empleado*" sí.

La otra pista: lo que aparece en las consultas de ejemplo del enunciado va en la cabeza. Es lo que te van a preguntar.

Un átomo también puede ir en la cabeza

Cuando la regla habla de un individuo específico, ponelo literal en la cabeza — no lo pidas con `=` en el cuerpo.

DA UNA VUELTA AL PEDO

```
puedeAndar(Sector, Empleado) :-  
    Sector = contaduria,  
    contador(Empleado),  
    honesto(Empleado).
```

PATTERN MATCHING

```
puedeAndar(contaduria, Empleado) :-  
    contador(Empleado),  
    honesto(Empleado).
```

Prolog unifica solo al hacer el matching de la cabeza, y encima el predicado queda inversible en las dos direcciones: podés preguntar `puedeAndar(contaduria, Quien)` y también `puedeAndar(Sector, lucia)`.

3 · Variables: ligadas, libres y puentes

TIPO	CÓMO SE RECONOCE	PARA QUÉ SIRVE
De la cabeza	Aparece en los parámetros del predicado	Entra o sale. Puede venir ligada (<code>tio(herbert, X)</code>) o libre (<code>tio(Q, W)</code>).

Puente	Solo en el cuerpo, dos o más veces	Une dos consultas. Es el corazón de casi toda regla no trivial.
Anónima _	Solo una vez, no te importa el valor	Consulta existencial: "existe algún X tal que..."

El puente, en detalle

```
tio(Tio, Sobrino) :-
    padre(Padre, Sobrino),    % ← nace acá: "el padre de Sobrino"
    hermano(Tio, Padre).      % ← se usa acá: "Tio es hermano de ESE padre"
```

Padre no es parámetro, no entra ni sale. Nace en la primera consulta y se consume en la segunda. Sin ella no tenés forma de decir "ese padre" en vez de "cualquier padre".

El error de reemplazarla por _ :

```
abuelo(Abuelo, Nieto) :- padre(Abuelo, _), padre(_, Nieto).
```

Esto dice: "Abuelo es padre de alguien, y alguien (otro cualquiera) es padre del Nieto". Los dos _ son variables **distintas** — no hay ninguna relación entre las dos consultas. Cualquier padre sería abuelo de cualquier chico.

Y el error simétrico: nombres distintos **no** garantizan valores distintos. Persona1 y Persona2 pueden tomar los dos el valor tomas . Si el enunciado dice "dos personas **distintas**", hay que pedirlo con \= explícitamente.

```
sonVecinos(P1, P2) :- viveEn(P1, Zona), viveEn(P2, Zona), P1 \= P2.
```

La misma variable repetida (Zona) fuerza igualdad. Variables distintas no fuerzan nada.

4 · Generador primero, restricción después

El orden de las consultas dentro de una regla **no es cosmético**. Cambia si el predicado anda o explota.

	GENERAN	SOLO VERIFICAN
Qué son	Hechos de la base y reglas construidas sobre ellos: <code>padre/2</code> , <code>viveEn/2</code> , <code>quedaEn/2</code> , <code>persona/1</code> ...	<code>not/1</code> , <code>forall/2</code> , <code>is/2</code> , <code>></code> , <code><</code> , <code>=<</code> , <code>>=</code> , <code>\=</code> , <code>length/2</code> sobre lista libre...
Con variable libre	Te van tirando candidatos uno por uno vía backtracking.	Fallan, devuelven vacío o tiran <i>"Arguments are not sufficiently instantiated"</i> .
Dónde van	Adelante.	Atrás.

Por qué falla, concretamente

```
bienUbicado(Persona) :- not(malUbicado(Persona)).
```

```
?- bienUbicado(Quien).           % ← no devuelve nada
```

Prolog llega a `not(malUbicado(Quien))` con `Quien` vacío y pregunta "¿es falso *malUbicado de nada?*". No sabe de quién le hablás. `not` sabe **verificar**, no sabe **buscar**.

```
bienUbicado(Persona) :-  
    viveEn(Persona, _),           % ← genera: fede, tomas, victoria...  
    not(malUbicado(Persona)).     % ← ahora sí puede verificar
```

Flujo: **generar** candidatos → **filtrar** los que no cumplen → **calcular** lo que haga falta.

Cómo elegir el generador. Preguntate: ¿qué hecho de la base me enumera *este tipo* de individuo?

- ¿Necesito zonas? → `quedaEn(_, Zona)` o `viveEn(_, Zona)`

- ¿Necesito personas? → `viveEn(Persona, _) , gana(Persona, _) , padre(Persona, _)`
- ¿Necesito destinos? → `quedaEn(Destino, _)`

Cuidado con cuál elegís: para `zonaDesierta` no sirve `viveEn`, justamente porque no vive nadie ahí. Tiene que ser `quedaEn`.

BONUS El generador además **acota el dominio**. Sin él, `llegaFacil(fede, pizza)` o `destinoAccesible(47)` darían `yes` para individuos que ni existen.

5 · "Todos": forall y la doble negación

Identificar el universo

Buscá la palabra **"todos"** y fijate a qué sustantivo modifica.

Ese sustantivo es el universo → antecedente. Lo que se afirma de él → consecuente.

```
forall( universo de discurso , condición que deben cumplir )
```

EJEMPLO TRABAJADO

"*difícilDeEstacionar* es verdadero para una zona habitada si **todos sus habitantes** tienen auto"

"Todos" modifica a *habitantes* → los habitantes son el universo.

MAL (INVERTIDO)

```
forall(tieneAuto(P),
       viveEn(P, Zona))
```

"Todos los dueños de auto de la ciudad viven en esta zona". No es lo que dice el enunciado.

BIEN

```
forall(viveEn(P, Zona),
       tieneAuto(P))
```

"Todos los que viven en esta zona tienen auto".

Pista de control: el antecedente casi siempre contiene la variable que *ya tenés ligada* (la de la cabeza), porque es lo que acota el universo a *este* caso. Si tu antecedente no la menciona, revisá.

La trampa de la verdad vacía

Si el antecedente genera un conjunto vacío, `forall` siempre da **true**. Una zona sin habitantes sería "difícil de estacionar"; un destino al que nadie quiere ir sería "accesible".

Solución: el generador de adelante, que además garantiza que el individuo existe y que el conjunto no está vacío.

```
difícilDeEstacionar(Zona) :-  
    viveEn(_, Zona),                                % existe y está habitada  
    forall(viveEn(Persona, Zona), tieneAuto(Persona)).
```

Cuando todavía no podés usar forall

"Todos cumplen" equivale a "**no existe ninguno que no cumpla**". Con dos negaciones te sale sin `forall` :

```
% 1) el contraejemplo  
malUbicado(Persona) :-  
    quiereIr(Persona, Destino),  
    viveEn(Persona, Zona),  
    not(quedaEn(Destino, Zona)).  
  
% 2) la negación del contraejemplo  
bienUbicado(Persona) :-  
    viveEn(Persona, _),  
    not(malUbicado(Persona)).
```

► Por qué funciona: el backtracking hace el trabajo del "todos"

OJO Fijate la diferencia entre los dos `not` del ejemplo:

- El de `malUbicado` recibe `quedaEn(racing, nuniez)` — **todo ligado** por las consultas anteriores. Puede verificar.
- El de `bienUbicado` recibiría `malUbicado(Quien)` con la variable libre. Por eso necesita el generador adelante.

6 · El test de 5 segundos

Antes de dar por terminado cualquier predicado, hacé estas tres consultas en la consola:

CONSULTA	QUÉ DETECTA	SI FALLA
<code>predicado(juan)</code> todo ligado	Que la lógica básica esté bien.	Revisá cabeza/cuerpo y los nombres de los predicados de la biblioteca.
<code>predicado(X)</code> variable libre	Inversibilidad. Es el que más puntos vale en el parcial.	Te falta un generador adelante, o tenés una restricción antes de tiempo.
<code>predicado(pizza)</code> algo que no existe	Dominio sin acotar y verdad vacía.	Falta el generador que garantice que el individuo existe.

7 · Diccionario castellano → Prolog

CUANDO EL ENUNCIADO DICE...	ESCRIBÍS...
"X es ... si ..."	Una regla: cabeza <code>:</code> - cuerpo
"... y ..."	Coma entre consultas
"... o ..."	Dos cláusulas del mismo predicado
" algún / existe un ..."	Variable anónima <code>_</code> o una variable libre en la consulta

" todos los A cumplen B"	<code>forall(A, B)</code> — o <code>not</code> del contraejemplo
" ninguno / nadie"	<code>not(consulta)</code>
"dos ... distintos "	<code>X \= Y</code> , al final
" el mismo / la misma"	Repetir la variable
" siempre puede / es"	Un hecho, sin cuerpo
"la cantidad de ..."	<code>findall + length/2</code>
"el total / la suma de ..."	<code>findall + sum_list/2</code> , o <code>maplist + sum_list</code>
" todas las combinaciones / qué posibilidades / podría llevar"	Explosión combinatoria: esqueleto de 3 cláusulas
"el máximo / el que más ..."	Generar el candidato y pedir con <code>forall</code> que ninguno lo supere

8 • Los 10 errores que se repiten

#	ERROR	CÓMO LO DETECTÁS
1	Cabeza y cuerpo invertidos	Leelo en voz alta con el "si". Suena al revés.
2	Cargar hechos en vez de escribir la regla	El enunciado dice "asumí que ya contás con..." y vos estás tipeando datos.
3	Redefinir un predicado que ya te dan	Estás pisando la base de conocimientos. Chequeá la pestaña Biblioteca.
4	Usar <code>_</code> donde hacía falta un puente	Dos consultas que deberían estar relacionadas y no comparten ninguna variable.
5	Asumir que <code>P1</code> y <code>P2</code> son distintos	Falta <code>\=</code> . Probá <code>predicado(juan, juan)</code> .
6	Restricción antes del generador	" <i>Arguments are not sufficiently instantiated</i> " o consulta vacía con variable libre.

7	Antecedente y consecuente del <code>forall</code> invertidos	El antecedente no menciona la variable de la cabeza.
8	Verdad vacía del <code>forall</code>	Un individuo inexistente o sin elementos da <code>yes</code> .
9	Repetir lógica en vez de delegar	El mismo par de consultas aparece en dos reglas distintas.
10	<code>Sector = ventas</code> en el cuerpo	Poné el átomo en la cabeza y listo.

Bonus: los code smells que la cátedra penaliza

- `findall` + `member` : armar una lista para después recorrerla de a uno. Contradicción algorítmica — consultá directo sobre la relación.
- `findall` **para ver si algo existe**: `findall(X, p(X), L), length(L, N), N > 0` . Usá `p(_)` o `not(p(_))` .
- **Modelar en negativo**: nunca escribas `noEsPasta(asado)` . Universo cerrado: lo que no está declarado es falso.
- **La regla gigante**: toda la lógica en una cláusula de ocho líneas en vez de subpredicados con nombre.

9 · Checklist de parcial

Antes de entregar cada punto, chequeá:

1. ¿Se lee como el enunciado, o hay que descifrarlo?
2. ¿Probé con **todas** las variables libres? (el enunciado casi siempre dice "el predicado debe ser inversible")
3. ¿Hay algún `not` , `forall` , `is` o comparación **antes** de su generador?
4. ¿Algún `forall` puede dar verdad vacía?
5. ¿Repetí lógica que ya tenía en otro predicado?
6. ¿Usé `findall` para algo que no sea una agregación real (sumar, contar)?
7. ¿Modelé algo en negativo?

8. ¿Perdí información del enunciado al modelar? (el pelaje y el color, no solo uno)

Reflejo para el parcial: si lees "todas las combinaciones", "qué posibilidades tenemos", "podría comprar / armar / llevar" → es **explosión combinatoria**. Andá directo al esqueleto de tres cláusulas: base, "lo elijo", "no lo elijo".

Los cinco pasos, en una línea: **frase en castellano** → **hecho o regla** → **clasificar variables** → **generador primero** → **delegar**.